

# A Proven Qualification Methodology for Embedded CMOS Memory Compilers

Remi CAPRARO, Lotfi BEN AMMAR

ATMEL Rousset  
Zone industrielle 13106 Rousset, France  
remi.capraro@rfo.atmel.com

**Abstract**—CMOS memory compilers become more and more the standard offer to fit customer needs for embedded applications. In order to select, verify, characterize and qualify such CMOS memory compilers, we have developed a complete and generic methodology. First step consists of the evaluation and the selection of the most relevant compilers. Then, we proceed to pre-silicon verifications and post-silicon characterization tests. Silicon characterization is a key step in the proposed methodology and is based on fabrication and test of a representative vehicle. Our approach has been proven on silicon for several technology nodes and process options, with different memory compiler providers. Some silicon results of our 130nm memory compilers are shown in this paper.

**Index Terms** — CMOS memory compilers, testchip design, silicon characterization, test patterns.

## I. INTRODUCTION

Nowadays, embedded CMOS memories are more and more used in system-on-chip (SoC) solutions [1, 2]. Chip designers need a large CMOS memory offer in terms of memory types, configurations, aspect ratios and features. Also, time-to-market constraints impose those IPs to be available and pre-qualified. Memory compilers are complex tools that offer such capabilities [3]. Thus, CMOS memory compilers tend to be the standard offer to satisfy customer requests for embedded applications.

Most of the time, although CMOS memory compilers are pre-qualified by providers, customers can't use them as they are in their own design flow. Compiler interfaces and generated views need to be modified by customers, in order to make them compatible with their design flow. That implies that several verifications have to be done, in order to guarantee quality of the offer [4]. Also, we often have the cases where on silicon characterization is either not done by the provider, or partially done using not up to date compiler versions. These are the different reasons why we developed and are using a dedicated and complete qualification methodology.

A global overview of our qualification methodology is presented in section II. Section III deals with CMOS memory compiler offer, memory features and the evaluation/selection approach. Pre-silicon verifications are presented in section IV. Regarding post-silicon part, contents and design of the characterization vehicle is presented in section V. Before concluding, test methodology is described in section VI, with

some silicon results for the qualification of 130nm memory compilers.

## II. METHODOLOGY OVERVIEW

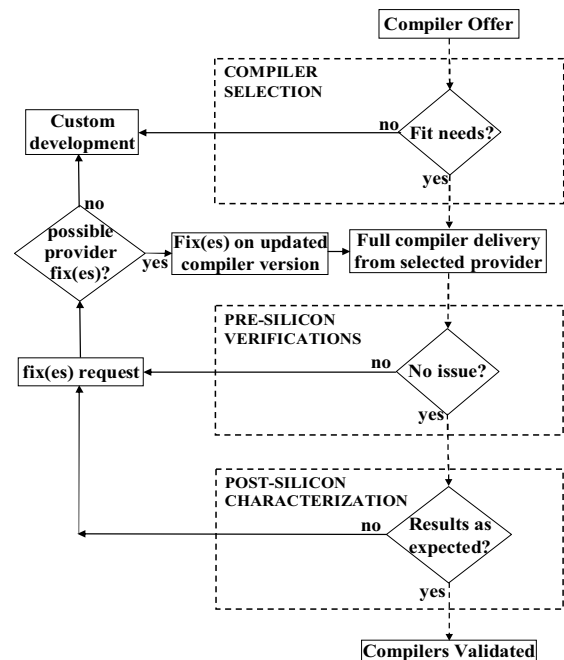


Figure 1: Qualification flow.

A complete and generic methodology for CMOS memory compiler qualification has been developed, as shown in Figure1. First step consists in evaluating memory compiler offer in order to select compilers that can satisfy customer needs. During the evaluation step, many IP providers are considered in order to compare their compiler features. The comparison takes in account several key parameters such as cost, flexibility, obtained performances and silicon status.

When compilers are selected, we proceed to pre-silicon verifications, then post-silicon characterization. Pre-silicon verifications consist in checking generated views and electrical performances [4]. Objective of this part is to have a maximum of guarantee before silicon production. Final step of the global methodology is the post-silicon characterization. It consists

firstly, in designing a dedicated testchip. Objective of this testchip is to verify memory functionality, design robustness and to measure memory performances on silicon and compare them with expected ones. For that, a complete test methodology needs to be developed. It notably consists in implementing test patterns dedicated to functional and yield tests [2, 5, 6, 7, 8, 9], and to memory performance measurements.

Note that in Figure 1, we also consider the cases where no compiler in the offer can satisfy customer needs or requested fixes can't be done by the provider. In both cases, compiler solution isn't relevant, and a custom development needs to be investigated.

In this paper, description of post-silicon characterization flow will be separated in two parts: characterization vehicle and test methodology.

### III. MEMORY COMPILER SELECTION

For a given technology node, CMOS memory compiler offer is very large in terms of providers and memory types. First objective of our methodology is to know which memory types are the most relevant for embedded applications and can fit customer needs. Then, we proceed to the selection of the most relevant provider. Compiler selection flow is presented in Figure 2.

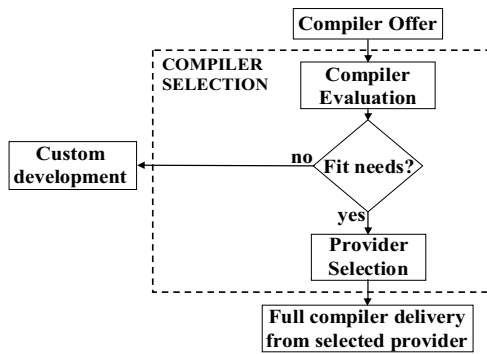


Figure 2: Compiler selection flow.

CMOS memory compilers are defined and characterized by several key features [3]. The most relevant ones are configuration range, optional features, global area, timing performances (access time, cycle time...), dynamic power consumption and leakage currents. Often, memory compilers include options which depend on provider and memory types. Here is a non-exhaustive list of options which can be found for CMOS memory compilers: asynchronous-write through, word-write mask, aspect ratio options, read margin adjustment, integrated BIST, power routing options, soft error repair and redundancy solutions.

So, first step of compiler selection flow is the selection of the most relevant compiler types for embedded applications. In general, most of embedded applications can be satisfied by the following five types: Single-Port RAM, Dual-Port RAM, Register File (1 and 2 ports) and ROM.

When compiler types are selected, next step consists in comparing features, performances and possible memory options of each provider and for each compiler type. No need to get full compilers for this comparison; only front-end views

from different providers are required. This evaluation needs to be done for a large set of memory configurations in order to have the best overview of memory performances.

Finally, when compiler types and provider are selected, we can go to the next step of the methodology: pre-silicon verifications.

### IV. PRE-SILICON VERIFICATIONS

Pre-silicon verifications allow to give a maximum of guarantee about functionality and performances, before silicon production. They are divided in three distinct steps: behavioral verifications, physical view checks and electrical simulations, as shown in Figure 3. In this section, we focus on the two last steps. Note that for this part, full compilers (all models and back-end views) are mandatory and must be delivered by the selected provider.

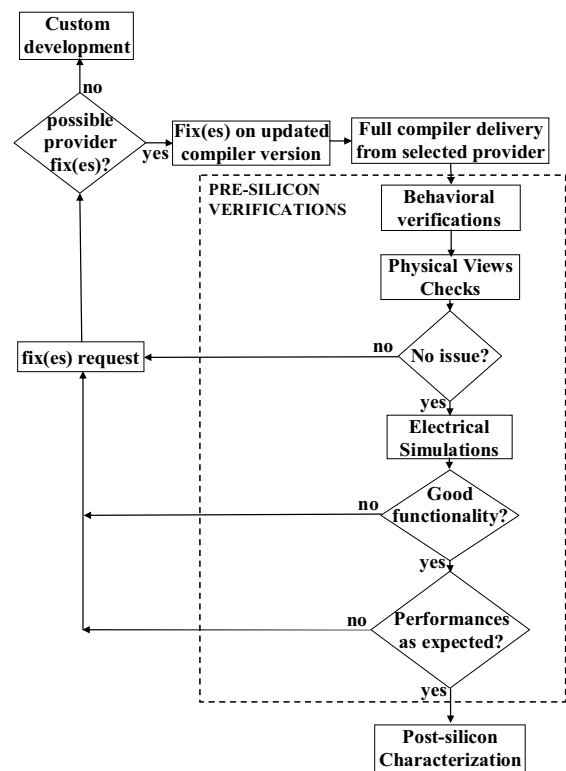


Figure 3: Pre-silicon verification flow.

For a given memory configuration, a compiler generates a physical view, generally called layout view. It represents all mask levels needed for silicon production of the memory. Such a view needs to be created with respect of design rules defined by the foundry, according to constraints of target technology process. So, first verifications consist in checking layout view with associated design rules. This step can be divided in three parts, generally called DRC (Design Rules Check), ARC (Antenna Rules Check) and ERC (Electrical Rules Check). For a given compiler type, a large set of memory configurations needs to be checked. Results of all of these verifications must present no error. If they do not, a fix request has to be addressed to the provider so that it performs some corrections on the concerned compiler. Such operations are repeated until results present no error.

Objective of the next step of physical view verifications is to check connectivity in the layout view. It consists of comparison between layout view and corresponding schematic view (or spice netlist). Such a schematic view is generated by memory compilers. Comparison operation is generally called LVS (Layout vs Schematic). A fix request is needed, if layout and schematic views don't match. In case of compiler fix, all physical view checks need to be redone until results present no error. When physical views are clean, we can continue with second part of pre-silicon verifications: electrical simulations.

Electrical simulations of a memory are performed on an extracted SPICE-like netlist. Such a netlist, extracted from layout view, represents the transistor level connectivity of the corresponding memory where capacitive and resistive parasitic elements are included. Netlist is simulated with SPICE-like simulator using representative transistor models. Note that for this part, it is not necessary to check a large set of memory configurations per compiler type. Minimum of five representative configurations per compiler type is considered sufficient.

First step of electrical simulation checks consists in verifying functionality of the memory. Dedicated stimuli need to be developed in order to explore all possible memory access modes and to exercise all possible options. Note that in case of malfunction, analysis of the root cause is preferable before requesting a fix to the provider. Second objective of electrical simulations is to check some specific electrical features (read margin, SNM...), in order to guarantee design robustness. Finally, the last step consists of the comparison of simulated electrical key parameters with datasheets ones. Key parameters to be checked are access time, cycle time, dynamic power consumption and leakage currents. Dedicated stimuli need to be developed in order to extract these electrical key parameters from electrical simulations. The provider is alerted in the case where memory performances are not as expected ones: an electrical re-characterization will be required.

Finally, when all back-end views verifications are clean and all electrical simulations give acceptable results for all checked memories of all compiler types, we can continue qualification process with post-silicon characterization flow, as shown in Figure 4.

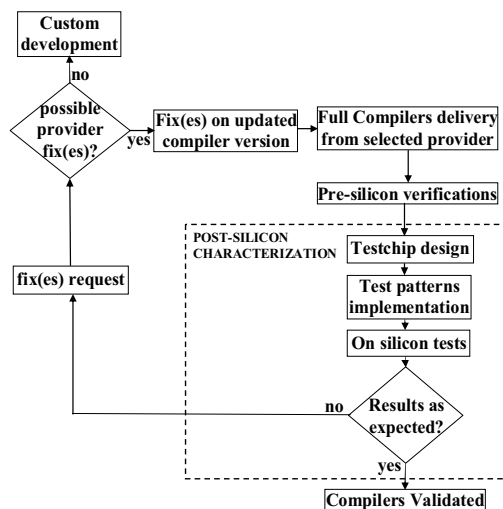


Figure 4: Post-silicon characterization flow.

## V. CHARACTERIZATION VEHICLE

The contents and the design of the dedicated testchip is a fundamental part of the methodology. Relevance and accuracy of silicon characterization results depend strongly on testchip design. Note that only one testchip needs to be fabricated to qualify all memory compilers, for a given provider and a given technology process.

### A. Testchip contents

Configuration of a memory corresponds to the number of rows and the number of columns in the bitcell array [3]. These two critical parameters are determined according to the number of words, the number of bits per word (data-bus) and the aspect ratio of the memory block. Examples of memory configurations are presented in Table 1.

| Number of words | Number of bits/word | Aspect ratio (*) | Number of rows | Number of columns | Relative footprint |
|-----------------|---------------------|------------------|----------------|-------------------|--------------------|
| 16K             | 32                  | 32               | 512            | 1024              | Wide               |
| 512             | 16                  | 4                | 128            | 64                | Tall               |
| 32              | 8                   | 2                | 16             | 16                | Square             |

(\*) aspect ratio is determined by the number of words per row.

Table 1. Examples of memory configurations.

The number of possible configurations generated by a compiler is very important (several hundreds). It's not possible to put all of them in the testchip. So, to be representative of compiler range, we select five instances per memory type/compiler (small, big, medium, tall and wide), as shown in Figure 5. Thus, we can accurately evaluate the impact of aspect ratio on measured electrical parameters.

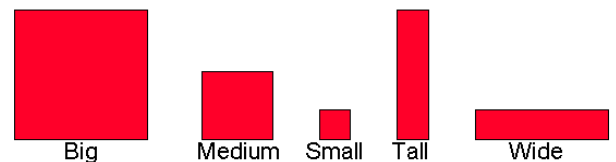


Figure 5. Set of five representative memory instances.

If selected compilers need to be qualified for different process options (ex: standard and high Vt options), we duplicate this set of memory instances on the same testchip. For example, for the qualification of four memory compilers for standard and high Vt options, that gives a total of 40 memory instances (5 memories \* 4 compilers \* 2 Vt options).

Moreover, most of the time, memory compilers allow to generate memory instances with specific options. These options can be dedicated either to yield improvement (read margin adjustment) or to increase memory flexibility (write mask, asynchronous-write-through...). Implementation of all possible options on all memories present in the testchip is recommended. If it is not possible, a minimum implementation of all possible specific options on only one memory per compiler is advised.

### B. Testchip Design

Design of the testchip must be very careful and robust, in order to get the simplest functionality and the most accurate measurements. So, design needs to be made so as to have a minimum functional risk and the best memory accessibility. We have seen that number of memory instances can be relatively important for such a testchip. It is not conceivable to have distinct I/O buses for each memory instance. Thus, a testchip interface with the minimum of possible I/O pads is targeted. One of possible solutions in order to fit with all these requirements is to have:

- All input signals (clock signal, addresses bus, data-in bus, write enable signal...) common to all memories.
- A multiplexing method for data-out bus. (see an example Figure 6).

In case of chip enable signal on memory interface, it would be judicious to implement a decoding logic in order to have only one memory active at the same time. So, for memory decoding (chip enable) and outputs multiplexing, further selection pins should be added.

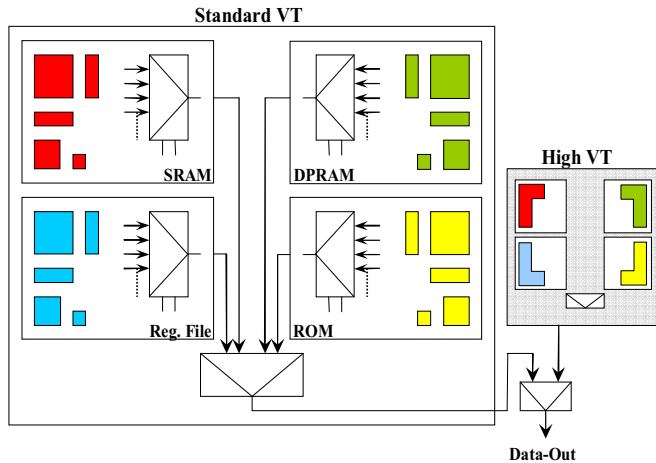


Figure 6. Example of Data-Out Multiplexing.

After the functional design, one of the most important parts of the testchip design in terms of safety and accuracy is the "Place & Route" step.

Concerning placement step (floor planning), memory instances have to be placed smartly in order to be easily connected to power pads and to have the most of possible physical orientations for a given compiler. A solution is to group memories according to their type, and to place them as close as possible to the pads (see an example Figure 7). In addition, for an optimized placement solution, we keep a dedicated place (preferably in the middle of testchip) to put all standard cells for glue logic.

Concerning interconnect routing and glue logic implementation, delay path constraints need to be set in order to optimize signal delays. For example, a minimum spread into each bus needs to be defined and set as a constraint in the routing flow.

Finally, post-PnR verifications need to be performed in order to validate design functionality, to check respect of

constraints and to extract some expected values to be compared with silicon ones. In general, these verifications consist in simulating HDL view from the post-PnR netlist with back-annotated timings.

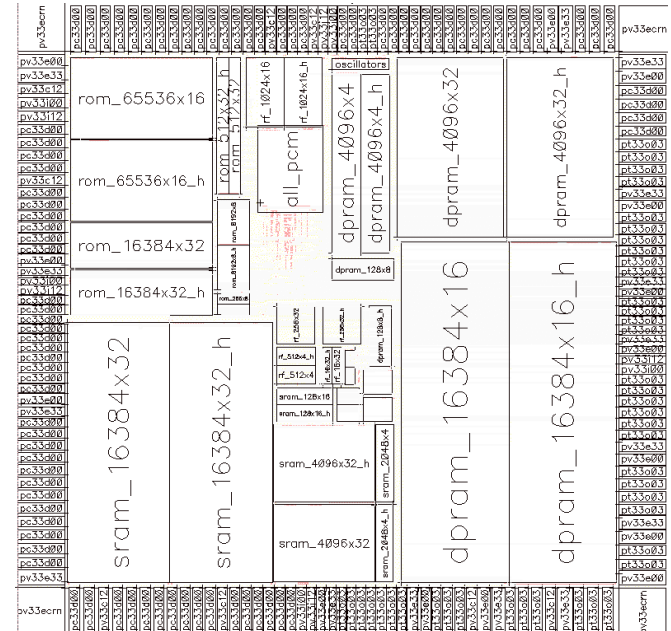


Figure 7. Example of Testchip Floor Planning.

### C. Specific Features

Specific features can be implemented in the testchip in order to improve characterization flow.

First of all, it is recommended to have some memories with separated supplies in order to get more accurate consumption measurements. For example, we take the biggest instance per compiler and per process option.

For timing measurements, the differential path method is implemented. Delays due to interconnects may be as important as intrinsic memory timings. In such case, micro-probing method can be implemented for more accurate measurements. That implies to put micro-probing plots into the testchip as close as possible from relevant memory pins.

In order to improve test and debug/engineering capabilities, addition of BIST/BISD modules for some memory instances is recommended [2].

## VI. TEST METHODOLOGY

This section is dedicated to the description of on silicon tests which are performed on the characterization vehicle. This section is divided in two parts. First part deals with functional and yield tests. Second step concerns measurement of memory performances.

### A. Functional & Yield Tests

Objective of functional and yield tests is to verify memory functionality, design robustness and yield results. Yield results

include global yield, yield for each memory compiler and for each memory instance.

Functional and yield results are obtained using open/short, Idd tests and dedicated test patterns. Dedicated test patterns need to be developed in order to get the best possible functionality coverage [2, 5, 6, 7, 8, 9]. A list of implemented dedicated test patterns is shown in Table 2.

| Test patterns description | Comments               |
|---------------------------|------------------------|
| Minimum Functional Test   |                        |
| Full Functional Test      |                        |
| March LR Test             | N.A. for ROM compilers |
| March SR Test             | N.A. for ROM compilers |
| March SS Test             | N.A. for ROM compilers |
| Data Retention Test       | N.A. for ROM compilers |
| Read Disturb Test         | N.A. for ROM compilers |
| BIST                      |                        |
| Cross-talk Test           | Multi-Port compilers   |
| Specific Multi-Port Test  | Multi-Port compilers   |
| ROM Sequential Read Test  | ROM compilers          |
| ROM Random Read Test      | ROM compilers          |

Table 2. Dedicated Functional & Yield Test Patterns.

Functional and yield tests need to be performed on a minimum of three representative wafers per lot. Such tests need to be also performed for standard and extended supply voltage and temperature ranges. Also, test at-speed is required [6, 10].

Shmoo plots (minimum supply voltage versus frequency) at different temperatures are also required. That allows to verify the design robustness, and to determine minimum supply voltage for functionality and maximum frequency.

### B. Performance Measurements

Objective of this part is to perform on silicon measurements in order to extract or calculate intrinsic memory performances to be compared to expected ones.

Here are key parameters relative to CMOS memories to be extracted from silicon tests:

- Access time
- Dynamic power consumption
- Leakage currents
- Minimum supply voltage for data retention
- Minimum supply voltage for functionality
- Maximum frequency

For both memory access time and dynamic power consumption measurements, dedicated test patterns need to be developed. Leakage currents can be measured at the end of any functional test patterns. Minimum supply voltage for data

retention can be determined using “Data Retention Test” dedicated patterns. Finally, minimum supply voltage for functionality and max frequency parameters can be extracted from shmoo plots, using March tests.

Performance measurements need to be performed for a minimum of three representative wafers and twenty dies per wafer. Measurements need to be also performed for standard and extended supply voltage and temperature ranges, in order to check performance dependencies.

### C. Experimental results for 130nm technology

Presented methodology has been proven over several technology nodes, for several memory compilers. Objective of this part is to show some silicon results that we have obtained for the qualification of 130nm memory compilers.

Firstly, you can find in Table 3, yield results for Two-Ports Register File compiler. Tests have been performed on three wafers, with twenty dies per wafer. Note that one die is considered as good, when all functional test patterns pass for all operating conditions.

| Compiler type | Freq.  | Process option | Wafer1 | Wafer2 | Wafer3 | Average |
|---------------|--------|----------------|--------|--------|--------|---------|
| 2P-RF         | 10MHz  | Std Vt         | 100%   | 95%    | 95%    | 96.67%  |
|               |        | High Vt        | 100%   | 90%    | 95%    | 95%     |
|               | 30 MHz | Std Vt         | 100%   | 95%    | 95%    | 96.67%  |
|               |        | High Vt        | 100%   | 90%    | 90%    | 93.33%  |

Table 3. 130nm yield results for 2P-RF compiler.

Shmoo plots are required in order to verify notably, memory robustness. In Figure 8, you can see an example of shmoo plot. Considering the aspect of this plot, we can conclude that the memory instance has a good behavior and an acceptable minimum supply voltage for functionality (800mV).

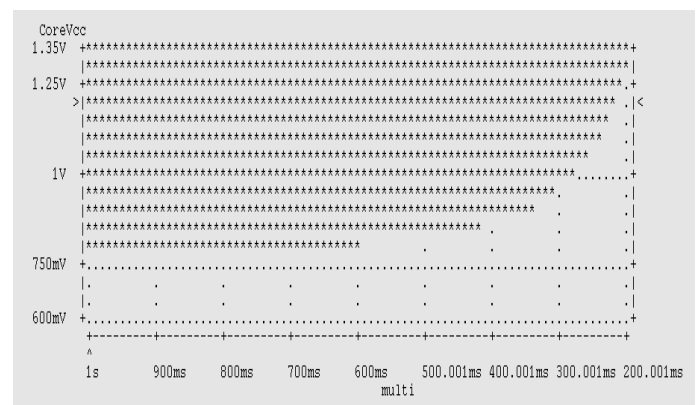


Figure 8. Shmoo plot for 130nm 1kx16 2P-RF.

Finally, on silicon measurements using characterization test patterns, allow to extract key parameters (access time, leakage currents...) and validate memory performances. In Table 4, you can find some results obtained for 1kx16 2P-RF instance.

|                              | RF-2P 1kx16<br>Std Vt |                 |
|------------------------------|-----------------------|-----------------|
| Key Parameter                | Expected Values       | Measured Values |
| Intrinsic memory access time | 2.8ns                 | 3.5ns           |
| Dynamic Power Consumption    | 7.5 $\mu$ W/MHz       | 9 $\mu$ W/MHz   |
| Leakage Currents             | 4.4 $\mu$ A           | 3.3 $\mu$ A     |

Table 4. Performance results for 130nm 1kx16 2P-RF.

## VII. CONCLUSION

A proven methodology to qualify CMOS memory compilers has been developed and presented in this paper. After selection of most relevant provider and compiler types, pre-silicon verifications need to be performed in order to guarantee quality of the offer, before silicon production. Then, an on silicon characterization flow has been demonstrated. A major step consists in designing a characterization vehicle. Also, dedicated test patterns need to be developed in order to have the best possible functionality coverage, and the most accurate performance measurements.

Our methodology has been proven over several technology nodes (0.25, 0.18 and 0.13 $\mu$ m), for several compiler providers and has given very good results. Some silicon results for qualification of 130nm memory compilers have been given in this paper. In spite of that, proposed methodology can constantly evolve in order to fit new requirements.

## REFERENCES

- [1] ITRS 2005 system Drivers, <http://public.itrs.net>.
- [2] S. Hamdioui, G. Gaydadjiev, Ad J. van de Goor, "The State-of-art and Futurer Trends in Testing Embedded Memories", in *IEEE International Workshop on Memory Technology, Design and Testing*, 2004, pp. 54-59.
- [3] G. Djaja, J. W. Nicholes, D. D. Smith, D. W. Knebelberger, G. W. Hancock, "Memory Compiler Interface and Methodology", United States Patent, 6405160, june 2002.
- [4] F. Renoux, "Let's Prove the Quality of our Memory Compilers", in *EE Times*, Sep. 1999 [online].
- [5] S. Hamdioui, Ad J. van de Goor, J. Delos, M. Rodgers, "Memory Test Experiment: Industrial Results and Data", *IEE Proc.-Comput. Digit. Tech.*, vol. 153, no 1, pp. 1-8, Jan 2006.
- [6] S. Hamdioui, Z. Als-Ars, Ad J. van de Goor, R. Wadsworth, "Impact of Stresses on the Fault Coverage of Memory Tests", in *IEEE International Workshop on Memory Technology, Design and Testing*, 2005, pp. 103-108.
- [7] Ad J. van de Goor, G. N. Gaydadjiev, V. G. Mikitjuk, V. N. Yarmolik, "March LR: A Test for Realistic Linked Faults", in *IEEE 14<sup>th</sup> VLSI Test Symposium*, 1996, pp 272-280.
- [8] S. Hamdioui, Ad J. van de Goor, M. Rodgers, "March SS: A Test for All Static Simple RAM Faults", in *IEEE International Workshop on Memory Technology, Design and Testing*, 2002, pp. 95-100.
- [9] S. Hamdioui, Ad J. van de Goor, "Efficient Tests for Realistic Faults in Dual-Port SRAMs", in *IEEE Transactions on Computers*, vol. 51, no 5, pp. 460-473, May. 2002.
- [10] A. K. Majhi, M. Azimane, G. Gronthoud, M. Lousberg, S. Eichenberger, F. Bowen, "Memory Testing Under Different Stress Conditions: An Industrial Evaluation", in *Proceedings of the 31st European Solid-State Circuits Conference*, 2005, pp. 299-302.